

Código

get_books

```
import re
import requests
from bs4 import BeautifulSoup
from concurrent.futures import ThreadPoolExecutor
import os

def get_links(n=-1):
    url = "https://www.gutenberg.org/browse/scores/top"
    try:
        response = requests.get(url)
        parser = BeautifulSoup(response.text, 'html.parser')

        ordered_list = parser.find('ol')
        list_items = ordered_list.find_all('li')

        if n != -1:
            indices = list(n) if isinstance(n, (list, range)) else [n]
            list_filtered = [list_items[i-1] for i in indices if i-1 < len(list_items)]
        else:
            list_filtered = list_items

        prefix = "https://www.gutenberg.org"
        suffix = ".txt.utf-8"

        links, titles = [], []

        for li in list_filtered:
            try:
                href = li.find("a").get("href")
                if href:
                    links.append(prefix + href + suffix)

                    # --- AQUÍ ESTÁ LA MAGIA DE LA LIMPIEZA ---
                    # 1. Obtenemos el texto
                    raw_title = li.get_text()
                    # 2. Reemplazamos CUALQUIER carácter que no sea letra/número por nada
                    clean_title = re.sub(r'^\w\s', '', raw_title)
                    # 3. Reemplazamos espacios por guiones bajos
                    title = re.sub(r'\s+', '_', clean_title)
                    # 4. Quitamos números finales
                    title = re.sub(r'_\d+$', '', title)
                    title += '.txt'
                    titles.append(title)
            except AttributeError:
                continue
```

```

        # Esta línea es la que faltaba o fallaba antes:
        return links, titles

    except Exception as e:
        print(f"Error: {e}")
        return [], []

def download_file(url, name):
    try:
        if not os.path.exists('libros'):
            os.makedirs('libros')
        file_path = os.path.join('libros', name)
        response = requests.get(url, stream=True)
        with open(file_path, mode='wb') as file:
            for chunk in response.iter_content(chunk_size=10240):
                file.write(chunk)
        print(f"Descargado: {name}")
    except Exception as e:
        print(f"Error en {name}: {e}")

def store_files(links, names):
    with ThreadPoolExecutor() as executor:
        executor.map(download_file, links, names)

def main(n=-1):
    links, titles = get_links(n)
    if links:
        print(f"Descargando {len(links)} libros...")
        store_files(links, titles)
        print("¡Listo!")
    else:
        print("No se encontraron enlaces.")

if __name__ == "__main__":
    n = range(1, 101)
    main(n)

```

clean_books

```
import os

def clean_content(text):
    # Marcadores comunes en Project Gutenberg
    start_markers = ["*** START OF", "***START OF"]
    end_markers = ["*** END OF", "***END OF"]

    start_idx = -1
    end_idx = -1

    # Buscar inicio
    for marker in start_markers:
        idx = text.find(marker)
        if idx != -1:
            # Buscar el salto de linea despues del marcador para no incluirlo
            start_idx = text.find('\n', idx) + 1
            break

    # Buscar fin
    for marker in end_markers:
        idx = text.rfind(marker)
        if idx != -1:
            end_idx = idx
            break

    # Si se encuentran ambos, recortar. Si no, devolver texto original.
    if start_idx != -1 and end_idx != -1:
        return text[start_idx:end_idx]
    return text

def main():
    folder = "libros"
    if not os.path.exists(folder):
        print(f"Carpeta {folder} no encontrada.")
        return

    files = os.listdir(folder)
    print(f"Limpiando {len(files)} libros...")

    for filename in files:
        if not filename.endswith(".txt"):
            continue

        path = os.path.join(folder, filename)
        try:
            with open(path, 'r', encoding='utf-8', errors='ignore') as f:
                content = f.read()
```

```
new_content = clean_content(content)

# Sobreescribimos el archivo con la versión limpia
with open(path, 'w', encoding='utf-8') as f:
    f.write(new_content)

except Exception as e:
    print(f"Error en {filename}: {e}")

print("Limpieza terminada.")

if __name__ == "__main__":
    main()
```

create_vocab

```
import os
import re

def get_stop_words():
    # Lista basica de stop words en ingles
    return set([
        "the", "of", "and", "to", "in", "a", "is", "that", "for", "it",
        "as", "was", "with", "be", "by", "on", "not", "he", "i", "his",
        "at", "are", "but", "have", "had", "which", "from", "this", "they",
        "you", "she", "or", "an", "were", "we", "their", "him", "been",
        "has", "there", "who", "will", "one", "all", "would", "her"
    ])

def process_books():
    folder = "libros"
    vocabulary = set()
    stop_words = get_stop_words()

    if not os.path.exists(folder):
        print("Carpeta no encontrada.")
        return

    files = [f for f in os.listdir(folder) if f.endswith(".txt")]
    print(f"Procesando vocabulario de {len(files)} libros...")

    for filename in files:
        path = os.path.join(folder, filename)
        try:
            with open(path, 'r', encoding='utf-8') as f:
                text = f.read()

                # 1. Transformar a minusculas
                text = text.lower()

                # 2. Limpieza de caracteres especiales (dejamos solo letras, numeros y espacios)
                text = re.sub(r'^a-z0-9\s', '', text)

                # 3. Tokenizacion (split por espacios)
                tokens = text.split()

                # 4. Eliminar stop words y agregar al vocabulario (set evita duplicados)
                for token in tokens:
                    if token not in stop_words and len(token) > 1:
                        vocabulary.add(token)

        except Exception as e:
            print(f"Error leyendo {filename}: {e}")
```

```
# Guardar vocabulario
with open("vocabulario.txt", "w", encoding="utf-8") as f:
    for word in sorted(vocabulary):
        f.write(word + "\n")

print(f"Vocabulario creado con {len(vocabulary)} palabras unicas.")
print("Guardado en 'vocabulario.txt'")

if __name__ == "__main__":
    process_books()
```

matrix_spark

```
import os
import re
from pyspark.sql import SparkSession
from pyspark.sql.functions import udf, col, lower, regexp_replace, split
from pyspark.sql.types import ArrayType, StringType, DoubleType
from pyspark.ml.feature import CountVectorizer, StopWordsRemover, Normalizer
from pyspark.mllib.linalg.distributed import RowMatrix
from pyspark.mllib.linalg import Vectors

def get_stop_words():
    # Mismas stop words que en el paso anterior para consistencia
    return [
        "the", "of", "and", "to", "in", "a", "is", "that", "for", "it",
        "as", "was", "with", "be", "by", "on", "not", "he", "i", "his",
        "at", "are", "but", "have", "had", "which", "from", "this", "they",
        "you", "she", "or", "an", "were", "we", "their", "him", "been",
        "has", "there", "who", "will", "one", "all", "would", "her"
    ]

def main():
    # Inicializar Spark
    spark = SparkSession.builder \
        .appName("GutenbergSimilarity") \
        .getOrCreate()

    sc = spark.sparkContext
    sc.setLogLevel("ERROR")

    # 1. Cargar libros
    # wholeTextFiles lee (ruta, contenido)
    books_rdd = sc.wholeTextFiles("libros/*.txt")

    # Convertir a DataFrame y limpiar ruta para dejar solo el nombre del archivo
    df = books_rdd.toDF(["path", "text"])
    df = df.withColumn("title", regexp_replace("path", ".*\\/", ""))

    # 2. Aplicar los 4 filtros (Minusculas, Regex, Tokenizacion, StopWords)

    # a. Minusculas y limpieza de caracteres (Regex)
    df_clean = df.withColumn("clean_text", lower(col("text")))
    df_clean = df_clean.withColumn("clean_text", regexp_replace("clean_text", "[^a-z0-9\\s]", ""))

    # b. Tokenizacion (split por espacio)
    df_clean = df_clean.withColumn("tokens", split(col("clean_text"), "\\s+"))

    # c. Eliminar Stop Words
```

```

    remover = StopWordsRemover(inputCol="tokens", outputCol="filtered",
stopWords=get_stop_words())
    df_filtered = remover.transform(df_clean)

    # 3. Vectorizacion (TF - Frecuencia de Termino)
    # Usamos vocabularioSize equivalente al que generaste o automatico
    vectorizer = CountVectorizer(inputCol="filtered", outputCol="raw_features",
vocabSize=10000, minDF=1.0)
    model = vectorizer.fit(df_filtered)
    df_vectorized = model.transform(df_filtered)

    # 4. Normalizacion (Para calcular Similitud del Coseno despues)
    normalizer = Normalizer(inputCol="raw_features", outputCol="features", p=2.0)
    df_norm = normalizer.transform(df_vectorized)

    # 5. Calculo de la Matriz de Similitud (Producto Punto de vectores normalizados)
    # Convertimos a RDD para usar operaciones de matriz eficientes
    mat_rdd = df_norm.select("features").rdd.map(lambda row:
Vectors.dense(row.features.toArray()))
    mat = RowMatrix(mat_rdd)

    # Compute column similarities (esto nos da similitud entre documentos si transponemos o A
* A^T)
    # Para 100 libros, es mas facil hacer producto punto localmente o via gramian
    # CoordinateMatrix es costoso, hacemos el producto punto manual que es equivalente a la
similitud coseno

    features = mat_rdd.collect()
    n = len(features)
    print(f"Calculando matriz {n}x{n}...")

    # Guardamos los titulos para referencia
    titles = df_norm.select("title").rdd.flatMap(lambda x: x).collect()

    sim_matrix = []
    for i in range(n):
        row = []
        vec1 = features[i]
        for j in range(n):
            vec2 = features[j]
            # Dot product de vectores normalizados = Similitud Coseno
            sim = vec1.dot(vec2)
            row.append(sim)
        sim_matrix.append(row)

    # 6. Guardar resultados para los siguientes pasos
    # Guardamos los titulos y la matriz en un archivo simple
    with open("matriz_similitud.txt", "w", encoding="utf-8") as f:
        # Primera linea: titulos separados por |
        f.write("|".join(titles) + "\n")

```

```
# Sigüientes líneas: filas de la matriz
for row in sim_matrix:
    f.write(",".join(map(str, row)) + "\n")

print("Matriz calculada y guardada en 'matriz_similitud.txt'")
spark.stop()

if __name__ == "__main__":
    main()
```

recommend

```
def load_data(filename="matriz_similitud.txt"):
    try:
        with open(filename, 'r', encoding='utf-8') as f:
            # La primera linea tiene los titulos separados por |
            titles_line = f.readline().strip()
            titles = titles_line.split('|')

            matrix = []
            for line in f:
                # Las demas lineas son los valores de similitud
                row = list(map(float, line.strip().split(',')))
                matrix.append(row)
            return titles, matrix
    except FileNotFoundError:
        print("Error: No se encontró el archivo de matriz.")
        return [], []

def get_recommendations(book_name, n=5):
    titles, matrix = load_data()

    if book_name not in titles:
        print(f"El libro '{book_name}' no se encuentra en la base de datos.")
        # Imprimimos los primeros 3 para que el usuario vea como se llaman
        if titles:
            print(f"Ejemplos de nombres válidos: {titles[:3]}")
        return []

    # Obtener el indice del libro
    book_idx = titles.index(book_name)

    # Obtener la fila de similitudes de ese libro
    scores = matrix[book_idx]

    # Crear lista de tuplas (indice, score) ignorando el mismo libro (donde score es > 0.99)
    # Usamos enumerate para guardar la posicion original
    ranked_books = []
    for idx, score in enumerate(scores):
        if idx != book_idx:
            ranked_books.append((idx, score))

    # Ordenar de mayor a menor similitud
    ranked_books.sort(key=lambda x: x[1], reverse=True)

    # Tomar los top N
    top_books = ranked_books[:n]

    results = [titles[idx] for idx, _ in top_books]
    return results
```

```
if __name__ == "__main__":  
    # Cargar titulos para tomar uno de ejemplo automaticamente  
    titles, _ = load_data()  
  
    if titles:  
        # Probamos con el primer libro de la lista  
        test_book = titles[0]  
        cant = 5  
  
        print(f"--- Recomendando libros basados en: {test_book} ---")  
        recomendaciones = get_recommendations(test_book, cant)  
  
        for i, libro in enumerate(recomendaciones, 1):  
            print(f"{i}. {libro}")  
    else:  
        print("No hay datos para procesar.")
```

summarize_book

```
import sys
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lower, regexp_replace, split
from pyspark.ml.feature import CountVectorizer, StopWordsRemover, IDF

def get_stop_words():
    return [
        "the", "of", "and", "to", "in", "a", "is", "that", "for", "it",
        "as", "was", "with", "be", "by", "on", "not", "he", "i", "his",
        "at", "are", "but", "have", "had", "which", "from", "this", "they",
        "you", "she", "or", "an", "were", "we", "their", "him", "been",
        "has", "there", "who", "will", "one", "all", "would", "her",
        "said", "project", "gutenberg"
    ]

def main():
    if len(sys.argv) < 3:
        print("Uso: python3 summarize_book.py <nombre_parcial_libro> <numero_palabras>")
        return

    target_book = sys.argv[1]
    try:
        n_words = int(sys.argv[2])
    except ValueError:
        print("El numero de palabras debe ser un entero.")
        return

    spark = SparkSession.builder.appName("GutenbergSummary").getOrCreate()
    spark.sparkContext.setLogLevel("ERROR")

    # 1. Carga y preprocesamiento
    df = spark.sparkContext.wholeTextFiles("libros/*.txt").toDF(["path", "text"])
    df = df.withColumn("title", regexp_replace("path", ".*\\/", ""))

    df_clean = df.withColumn("clean_text", lower(col("text")))
    df_clean = df_clean.withColumn("clean_text", regexp_replace("clean_text", "[^a-z0-9\\s]", ""))
    df_clean = df_clean.withColumn("tokens", split(col("clean_text"), "\\s+"))

    remover = StopWordsRemover(inputCol="tokens", outputCol="filtered",
stopWords=get_stop_words())
    df_filtered = remover.transform(df_clean)

    # 2. Calculo de TF-IDF
    cv = CountVectorizer(inputCol="filtered", outputCol="raw_features", vocabSize=20000,
minDF=1.0)
    cv_model = cv.fit(df_filtered)
    df_tf = cv_model.transform(df_filtered)
```

```

idf = IDF(inputCol="raw_features", outputCol="features")
idf_model = idf.fit(df_tf)
df_tfidf = idf_model.transform(df_tf)

# 3. Extraccion de palabras clave para el libro solicitado
book_row = df_tfidf.filter(col("title").contains(target_book)).first()

if not book_row:
    print(f"No se encontro ningun libro que contenga: {target_book}")
    spark.stop()
    return

tfidf_vector = book_row["features"]
vocabulary = cv_model.vocabulary

# Unir indices con palabras y ordenar por relevancia
word_scores = []
for idx, score in zip(tfidf_vector.indices, tfidf_vector.values):
    word_scores.append((vocabulary[idx], score))

word_scores.sort(key=lambda x: x[1], reverse=True)

print(f"\n--- Palabras resumen para: {book_row['title']} ---")
for i in range(min(n_words, len(word_scores))):
    word, score = word_scores[i]
    print(f"{i+1}. {word}")

spark.stop()

if __name__ == "__main__":
    main()

```